

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Vishwa.U	Reg. No.:	192511192
Section / Batch:	Slot-B-2025	Date:	26.08.2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
 - Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct

I Vishwa.U(192511192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator Faculty Incharge

Q1. I/O mechanism for real-time embedded system

Problem Understanding: Design an I/O communication scheme for a real-time system where CPU load must stay under 15%, several sensors interrupt at a 1 kHz rate (every 1 ms), vectored interrupts are mandated, and the addressing mode (memory-mapped vs I/O-mapped) must be chosen for lowest latency.

Constraint Analysis (priority order):

1. **Real-time latency** – implicit top priority; governs the interrupt architecture.
2. **CPU utilization $\leq 15\%$** – hard budget: only $0.15 \times 1 \text{ ms} = 150 \mu\text{s}$ of ISR time per 1 ms period is allowed.
3. **1 kHz interrupt rate** – sets the timing grid every ISR must fit inside.
4. **Vectored interrupts required** – removes the need for software polling of status registers.
5. **Addressing mode** – affects per-access latency of every register read/write inside the ISR.

Solution:

- Use a **Vectored Interrupt Controller (VIC)**: each sensor gets a unique vector address, so hardware jumps straight to its ISR — $O(1)$ dispatch instead of $O(n)$ polling.
- Assign **priority levels** to sensors by criticality; allow preemption of lower-priority ISRs.
- Choose **memory-mapped I/O**: sensor registers sit in normal address space, so ISRs use ordinary load/store instructions, benefit from burst/cache-friendly access, and avoid the extra bus-cycle/decode overhead of isolated I/O-mapped IN/OUT instructions.
- Split each ISR into a **short top half** (acknowledge + latch data into buffer) and a **deferred bottom half** (heavy processing), keeping ISR execution time minimal.

Application of Concepts (calculation):

- Interrupt period $T = 1 \text{ ms}$; budget = $150 \mu\text{s}$ total ISR time per period.
- Example: $10 \text{ sensors} \times 15 \mu\text{s each} = 150 \mu\text{s} \rightarrow$ exactly meets the 15% ceiling.
- Vectored dispatch overhead $\approx$ a few clock cycles vs. $O(n)$ polling overhead that grows with sensor count.

Justification: Vectored dispatch + memory-mapped I/O + split ISR design together minimize per-event overhead, keeping worst-case CPU time inside the 15% budget even as sensor count grows — a polled or I/O-mapped design would not scale the same way.

Q2. High-speed storage I/O subsystem (NVMe/PCIe)

Problem Understanding: Build an I/O subsystem for storage exceeding 3 GB/s using NVMe over PCIe, where the CPU must not perform bulk data movement, and DMA + interrupt notification handle completion.

Constraint Analysis:

1. **Throughput > 3 GB/s** → link must be sized correctly (PCIe3 $\times 4 \approx 3.94$ GB/s, or PCIe4 $\times 2 \approx 3.94$ GB/s).
2. **NVMe mandated** → needed for deep parallel command queues that AHCI/SATA cannot sustain at this rate.
3. **CPU excluded from bulk transfer** → requires bus-mastering DMA, not programmed I/O.
4. **Completion notification** → interrupts, not CPU polling.

Solution:

- **PCIe Gen4 $\times 4$** link (headroom above the 3 GB/s floor).
- NVMe's **multiple submission/completion queues** (up to 64K queues $\times$ 64K commands) allow deep, parallel command dispatch across cores.
- **Bus-mastering DMA engine** in the NVMe controller: CPU only builds a descriptor (source, destination, length); the controller moves data directly SSD $\leftrightarrow$ RAM.
- **MSI-X interrupts**, one vector per queue, with **interrupt coalescing** (batch completions before raising an interrupt) to control interrupt rate at high throughput.

Application of Concepts:

- Lane check: $3 \text{ GB/s} \div \sim 0.985 \text{ GB/s per PCIe3 lane} \approx 3$ lanes minimum → use $\times 4$ for margin.
- DMA transfer of 128 KB at 3.94 GB/s $\approx 32.5 \mu\text{s}$ — versus PIO, which would need $\sim 32,768$ CPU load/store iterations for the same block, clearly breaking any CPU budget.
- Without coalescing, at 4 KB/completion: $3 \text{ GB/s} \div 4 \text{ KB} \approx 750,000$ interrupts/s (infeasible). Coalescing every 32 completions → $\approx 23,000$ interrupts/s (workable).

Justification: PCIe lane sizing satisfies raw throughput; bus-mastering DMA satisfies "no CPU bulk transfer"; MSI-X + coalescing gives low-overhead completion signaling — together meeting every constraint simultaneously.

Q3. Multi-device (USB + Thunderbolt) communication architecture

Problem Understanding: Connect 6 USB peripherals and 2 Thunderbolt devices while avoiding interrupt starvation, using both hardware and software interrupts, with fair bus arbitration.

Constraint Analysis:

1. USB devices **share one bus** (contention-prone); Thunderbolt devices are **point-to-point/PCIe-tunneled** (largely contention-free between themselves, but share host PCIe/interrupt resources).
2. **Starvation risk** if high-priority or bulk traffic dominates arbitration.
3. **HW + SW interrupt split** required — HW for real completion events, SW for deferred processing.
4. **Fairness** must balance isochronous, interrupt, bulk, and control traffic types.

Solution:

- **USB arbitration:** host controller reserves bandwidth per USB frame (1 ms) / microframe (125 μ s): Control gets a guaranteed small slice, Interrupt transfers get guaranteed periodic polling slots, Isochronous gets a reserved percentage, and Bulk uses leftover bandwidth with round-robin among the bulk devices so none monopolizes it.
- **Thunderbolt arbitration: weighted round-robin (WRR)** at the Thunderbolt switch, with bandwidth reservation for latency-sensitive tunnels and fair sharing of the remainder between the 2 devices.
- **Interrupt handling:** hardware interrupts only acknowledge and queue work (top half); **software interrupts/soft-IRQs** perform the actual data processing in a schedulable context (bottom half) — this prevents a burst from one bus starving others, since the OS scheduler fairly time-slices the deferred work.
- Add **aging**: any device's deferred task waiting abnormally long gets a temporary priority boost, bounding worst-case wait time.

Application of Concepts:

- USB link budget: even 6 low/full-speed devices typically consume a small fraction of USB2/3 bandwidth, leaving headroom.
- Thunderbolt (20–40 Gbps per port) can give each of the 2 devices a guaranteed minimum with dynamic sharing of the rest.
- This is the standard two-level "hardirq / softirq" model used in real operating systems.

Justification: Reservation-based arbitration (USB) + WRR (Thunderbolt) + strict HW/SW interrupt separation + aging directly targets and eliminates starvation while remaining fair across heterogeneous device classes.

Q4. Data acquisition system (500 MB/s stream)

Problem Understanding: Continuously acquire data at 500 MB/s with minimal CPU intervention; explicitly compare DMA vs interrupt-driven I/O; use memory-mapped I/O for device registers.

Constraint Analysis:

- 1. Sustained 500 MB/s **rules out programmed I/O** — the CPU cannot copy data word-by-word at this rate and stay "minimal."
- 2. The problem **requires a quantitative DMA vs. interrupt-driven comparison.**
- 3. Device control/status registers should be **memory-mapped** for simple, fast configuration access.

Solution & Comparison:

- **Interrupt-driven I/O per small unit** is shown (below) to be infeasible at this data rate.
- **DMA-based streaming:** a DMA controller moves data directly device → memory in large bursts into a **double-buffered (ping-pong) circular buffer**; the CPU is interrupted only once per buffer swap, not per sample.
- **Memory-mapped registers** (buffer base address, size, control/status, threshold) let firmware configure the DMA once with ordinary loads/stores; no polling needed in steady state.

Application of Concepts (the required comparison):

Approach	Interrupt granularity	Interrupts/sec	CPU overhead (at 1 μs/ISR)
Interrupt-driven	4 bytes	≈125,000,000/s	Impossible (<8 ns/interrupt needed)
Interrupt-driven	1 KB	≈500,000/s	≈50% CPU — violates "minimal"
DMA	64 KB/block	≈7,629/s	≈0.76% CPU
DMA	256 KB/block	≈1,907/s	≈0.19% CPU

Justification: The numbers prove interrupt-driven I/O cannot sustain 500 MB/s without overwhelming the CPU, while block-based DMA reduces interrupt frequency by 2–3 orders of magnitude. Memory-mapped registers keep the (rare) control-path operations simple. This hybrid — memory-mapped control + DMA data path + coarse-grained interrupts — is the standard optimal architecture for high-throughput acquisition.

Q5. Interrupt handling mechanism (nested, <10 μ s latency)

Problem Understanding: Design interrupt handling that supports nesting, guarantees worst-case latency under 10 μ s, uses vectored interrupts for high-priority devices, and software interrupts for background tasks.

Constraint Analysis:

1. **Nesting** requires a priority-based controller where a higher-priority interrupt can preempt a running lower-priority ISR.
2. **<10 μ s latency** is a hard real-time ceiling on recognition + context-save + dispatch, even under preemption.
3. **Vectored interrupts for high priority** $\rightarrow$ needed to meet the tight latency via direct $O(1)$ dispatch.
4. **Software interrupts for background work** must never delay hardware-critical interrupts.

Solution:

- **Multi-level priority vectored controller** (ARM NVIC-style): each priority level has its own vector entry and can preempt any lower level; equal/higher levels are masked appropriately to avoid re-entrancy bugs.
- High-priority devices get **top priority levels + vectored dispatch** — hardware auto-saves minimal context and jumps directly via the vector table.
- **Tail-chaining**: if a higher-priority interrupt arrives while another is being dispatched, the controller reuses the in-progress context save instead of fully unwinding, cutting back-to-back overhead.
- **Software interrupts** (SWI/trap or OS soft-IRQs) are pinned to the **lowest priority** — used only for deferred, non-critical work (logging, scheduling) — and run only in gaps between hardware activity, so they can never inflate hardware latency.
- Keep high-priority ISRs minimal (register read/write + flag clear only); defer heavier work to software interrupts.

Application of Concepts (latency budget, must total <10 μ s):

Stage	Estimated time
Interrupt propagation/sync	$\sim 0.5 \mu$ s
Hardware context save	$\sim 2 \mu$ s
Vector fetch + jump	$\sim 0.2 \mu$ s
ISR prologue	$\sim 1 \mu$ s
Subtotal	$\sim 3.7 \mu$s (comfortable margin under 10 μ s)

Nesting adds $\sim 2\text{--}3 \mu$ s per additional legitimate preemption level, still within budget since only the top-priority chain is time-critical.

Justification: Hardware-vectored, priority-nested dispatch satisfies the nesting requirement; minimal ISRs plus tail-chaining keep worst-case latency well under 10 μ s; strictly bottom-ranked software interrupts guarantee they never compete with hardware-critical paths. This layered design mirrors real controllers (ARM Cortex-M NVIC, x86 APIC).